Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Конструирование программ»

ОТЧЕТ
к лабораторной работе №6
на тему:
«ЗАГРУЗКА И ВЫПОЛНЕНИЕ ПРОГРАММ. РАБОТА С ПАМЯТЬЮ»
БГУИР 6-05-0612-02 113

Выполнил студент группы 453503
ХАЛАМОВ Николай Андреевич

_____
(дата, подпись студента)

Проверил ассистент каф. Информатики
РОМАНЮК Максим Валерьевич

_____
(дата, подпись преподавателя)

Минск 2025

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Задание 1. Вариант 4. Написать программу, запускающую другую программу N раз (N – число в диапазоне [1, 255]). Имя запускаемой программы задается в текстовом файле в K строке (K – число в диапазоне [1, 255]).

2 ВЫПОЛНЕНИЕ РАБОТЫ

Целью данной лабораторной работы являлось изучение механизмов загрузки и выполнения программ в операционной системе DOS, а также освоение методов управления памятью. В рамках работы была разработана программа, которая запускает другую программу заданное количество раз. Имя запускаемой программы и её параметры считываются из текстового файла по указанному номеру строки. Программа принимает три параметра командной строки: N – количество запусков (1-255), K – номер строки в файле (1-255) и имя текстового файла. Реализация включает полную обработку ошибок с выводом информационных сообщений в консоль, парсинг командной строки как основной программы, так и дочерней, а также корректное управление памятью для обеспечения возможности загрузки других программ. В процессе выполнения работы были изучены функции DOS для управления памятью (INT 21h/4Ah), загрузки программ (INT 21h/4Bh), работы с файлами и формирование блока параметров EXEC для передачи аргументов командной строки дочернему процессу.

1. Освобождение памяти для загрузки дочерней программы

При запуске EXE-программы операционная система DOS выделяет ей всю доступную память, что препятствует загрузке других программ. Для решения этой проблемы необходимо использовать функцию DOS 4Ah (изменение размера блока памяти), которая уменьшает выделенный программе блок до минимально необходимого. Реализация начинается с получения сегментного адреса PSP (Program Segment Prefix), после чего вычисляется реальный размер программы в параграфах (16-байтных блоках) путём деления смещения метки конца программы на 16 с округлением вверх. К полученному значению добавляется запас для PSP и стека. Важно отметить, что ES должен указывать на PSP при вызове функции 4Ah, иначе произойдёт ошибка. После успешного изменения размера блока сегментные регистры восстанавливаются для корректной работы с данными программы.

Листинг кода 1 – Освобождение памяти для загрузки дочерней программы

```
start:
   mov ax, @data
   mov ds, ax
```

```
    mov ah, 62h          ; Get PSP segment
    int 21h
    mov es, bx           ; ES -> PSP

    mov bx, offset program_end
    add bx, 15
    mov cl, 4
    shr bx, cl           ; BX = size in paragraphs
    add bx, 10h + 10h    ; Add PSP + stack reserve

    mov ah, 4Ah          ; Resize memory block
    int 21h

    mov ax, @data
    mov es, ax
```

2. Парсинг командной строки основной программы

Парсинг командной строки осуществляется путём чтения данных из PSP по смещению 80h, где первый байт содержит длину командной строки, а со смещения 81h начинаются сами параметры. Программа последовательно извлекает три параметра: два числовых значения N и K, а затем имя файла. Для пропуска пробелов и табуляций используется вспомогательная процедура skip_spaces, которая устанавливает флаг переноса при достижении конца строки. Числовые параметры преобразуются из ASCII-представления в двоичное с помощью процедуры parse_number, которая умножает накопленное значение на 10 и добавляет очередную цифру. После извлечения каждого параметра выполняется проверка на корректность: числа должны быть больше нуля, а имя файла не должно быть пустым. В случае ошибки процедура возвращает код 1 в регистре AX, что приводит к выводу сообщения об ошибке и завершению программы.

Листинг кода 2 – Парсинг командной строки основной программы

```
parse_cmdline proc
    push bx
    push cx
    push dx
    push si
    push di

    mov ah, 62h
    int 21h
    mov es, bx

    mov si, 80h
    mov cl, es:[si]
    cmp cl, 0
    je pc_error
    inc si
```

```
    call skip_spaces
    jc pc_error

    call parse_number
    jc pc_error
    mov N_val, al
    cmp al, 0
    je pc_error

    call skip_spaces
    jc pc_error

    call parse_number
    jc pc_error
    mov K_val, al
    cmp al, 0
    je pc_error

    call skip_spaces
    jc pc_error

    lea di, fileListName

pc_copy_fname:
    cmp cl, 0
    je pc_finish_name
    mov al, es:[si]
    cmp al, ' '
    je pc_finish_name
    cmp al, 09h
    je pc_finish_name
    cmp al, 0Dh
    je pc_finish_name
    mov [di], al
    inc di
    inc si
    dec cl
    jmp pc_copy_fname

pc_finish_name:
    mov byte ptr [di], 0
    cmp di, offset fileListName
    je pc_error

    xor ax, ax
    jmp pc_end

pc_error:
    mov ax, 1

pc_end:
    pop di
    pop si
    pop dx
```

```
    pop cx
    pop bx
    ret
parse_cmdline endp
```

3. Чтение K-й строки из файла и парсинг параметров дочерней программы

Чтение данных из файла реализовано посимвольно с использованием функции DOS 3Fh. Файл открывается функцией 3Dh в режиме только для чтения, после чего программа последовательно считывает символы до обнаружения символа перевода строки (CR, код 0Dh). Счётчик строк увеличивается при каждом переводе строки и сравнивается с искомым номером K. При нахождении нужной строки вызывается процедура parse_program_line, которая разделяет строку на две части: имя программы (до первого пробела) и аргументы командной строки (всё остальное). Имя программы сохраняется в буфер progName с двумя терминаторами: нулевым байтом для функции EXEC и символом '$' для функции вывода DOS 09h. Аргументы копируются в отдельный буфер cmd_args. Такое разделение необходимо для последующего формирования командной строки в формате DOS PSP. В случае если файл не удаётся открыть или K-я строка не найдена, процедура возвращает соответствующий код ошибки.

Листинг кода 3 – Чтение K-й строки из файла и парсинг параметров дочерней программы

```
parse_program_line proc
    push ax
    push si
    push di

    call clear_progName
    call clear_cmd_args

    lea si, line_buf
    lea di, progName

copy_prog:
    mov al, [si]
    cmp al, 0
    je parse_done
    cmp al, ' '
    je found_space
    cmp al, 09h
    je found_space
    mov [di], al
    inc si
    inc di
    jmp copy_prog
```

```
found_space:
    mov byte ptr [di], 0
    inc di
    mov byte ptr [di], '$'

skip_spaces_after_prog:
    inc si
    mov al, [si]
    cmp al, ' '
    je skip_spaces_after_prog
    cmp al, 09h
    je skip_spaces_after_prog
    cmp al, 0
    je parse_done

    lea di, cmd_args
copy_args_loop:
    mov al, [si]
    cmp al, 0
    je args_done
    mov [di], al
    inc si
    inc di
    jmp copy_args_loop

args_done:
    mov byte ptr [di], 0

parse_done:
    pop di
    pop si
    pop ax
    ret
parse_program_line endp
```

4. Формирование командной строки для дочерней программы

Командная строка для дочерней программы должна быть сформирована в специальном формате DOS PSP: первый байт содержит длину строки (без учёта самого байта длины и символа CR), затем следует пробел, сами аргументы и символ возврата каретки (CR, 0Dh). Процедура build_cmd_tail сначала вычисляет длину аргументов путём последовательного подсчёта символов до нулевого терминатора. Если аргументы отсутствуют, формируется пустая командная строка из двух байт: 0 и 0Dh. В противном случае в первый байт записывается длина строки увеличенная на единицу (для учёта обязательного пробела), затем записывается пробел, копируются все аргументы из буфера cmd_args и в конец добавляется символ CR. Этот формат соответствует стандарту DOS и позволяет дочерней программе корректно

получить все переданные параметры через стандартный механизм доступа к командной строке.

Листинг кода 4 – Формирование командной строки для дочерней программы

```
build_cmd_tail proc
    push ax
    push si
    push di
    push cx

    lea di, cmd_tail
    mov cx, 128
    xor al, al
clear_tail:
    mov [di], al
    inc di
    loop clear_tail

    lea si, cmd_args
    xor cx, cx
count_len:
    mov al, [si]
    cmp al, 0
    je len_done
    inc cx
    inc si
    jmp count_len

len_done:
    cmp cx, 0
    je empty_tail

    lea di, cmd_tail
    inc cx
    mov [di], cl
    inc di
    mov byte ptr [di], ' '
    inc di

    lea si, cmd_args
copy_args_to_tail:
    mov al, [si]
    cmp al, 0
    je tail_done
    mov [di], al
    inc si
    inc di
    jmp copy_args_to_tail

tail_done:
    mov byte ptr [di], 0Dh
    jmp build_exit
```

```
empty_tail:
    lea di, cmd_tail
    mov byte ptr [di], 0
    inc di
    mov byte ptr [di], 0Dh

build_exit:
    pop cx
    pop di
    pop si
    pop ax
    ret
build_cmd_tail endp
```

5. Загрузка и выполнение дочерней программы

Загрузка программы осуществляется через функцию DOS 4Bh с подфункцией 00h (загрузить и выполнить). Перед вызовом необходимо инициализировать блок параметров EXEC, который содержит указатели на сегмент окружения (0 означает наследование текущего окружения), командную строку и два FCB (File Control Block). Регистровая пара DS:DX должна указывать на ASCIIZ-строку с именем программы, а ES:BX на блок параметров. Критически важно сохранить регистры DS и ES перед вызовом функции, так как DOS может их изменить в процессе загрузки и выполнения дочерней программы. После возврата управления необходимо немедленно восстановить сегментные регистры на сегмент данных основной программы, иначе любое обращение к данным приведёт к ошибке. Проверка флага переноса (CF) после вызова INT 21h позволяет определить успешность операции и в случае ошибки вернуть соответствующий код для обработки.

Листинг кода 5 – Загрузка и выполнение дочерней программы

```
exec_program proc
    push bx
    push dx
    push ds
    push es

    push ds
    pop es

    lea dx, progName
    mov ax, 4B00h
    mov bx, offset ExecEnv

    int 21h
    jc ep_error

    mov ax, @data
    mov ds, ax
```

```
    mov es, ax

    xor ax, ax
    jmp ep_exit

ep_error:
    mov ax, @data
    mov ds, ax
    mov es, ax

    mov ax, 1

ep_exit:
    pop es
    pop ds
    pop dx
    pop bx
    ret
exec_program endp
```

ВЫВОД

В результате выполнения лабораторной работы была успешно разработана программа на языке ассемблер, демонстрирующая механизмы загрузки и выполнения программ в операционной системе DOS. Программа корректно обрабатывает командную строку, считывает данные из файла, формирует параметры для дочерней программы и осуществляет её многократный запуск. Особое внимание было уделено управлению памятью: реализовано освобождение избыточной памяти через функцию DOS 4Ah, что является критически важным условием для возможности загрузки других программ. Изучены особенности работы с блоком параметров EXEC, включая формирование командной строки в формате DOS PSP и передачу параметров дочернему процессу. Реализована полная обработка ошибок на всех этапах: при парсинге командной строки, открытии файла, чтении данных и запуске программы. Все ошибочные ситуации сопровождаются информативными сообщениями, что соответствует требованиям задания. Программа демонстрирует понимание низкоуровневых механизмов операционной системы DOS, включая работу с PSP, управление сегментными регистрами и взаимодействие с системными функциями через программные прерывания. Полученные знания и навыки могут быть применены при разработке системного программного обеспечения и понимании принципов работы современных операционных систем.